

Visual Studio 2013/2015 & Langage C#

Premiers pas en .NET

SOMMAIRE

<u>GENERALITES SUR LES APPLICATIONS FENETREES.</u>	<u>3</u>
<u>ETAPE 1 : DESSINER LES ECRANS.</u>	<u>4</u>
<u>ETAPE 2 : DEFINIR LES PROPRIETES DE CHAQUE CONTROLE.</u>	<u>12</u>
<u>ETAPE 3 : CODER LES PROCEDURES EVENEMENTIELLES</u>	<u>14</u>
<u>LANCER L'APPLICATION ET CONTROLER LES RESULTATS</u>	<u>18</u>
<u>SYNTHESE ; LES POINTS-CLES DE L'EXERCICE</u>	<u>19</u>

Généralités sur les applications fenêtrées.

C# est un langage qui permet de développer des applications Windows (et même Web, mais c'est une autre histoire...), semblables à celles que vous voyez partout sur micro-ordinateurs. L'interface graphique d'une application Windows en C# comporte un ou plusieurs « écran / formulaire » dénommés feuilles (*forms*) dans C#. Chaque écran comporte en général des champs de saisie et des champs d'affichage, ainsi que un ou plusieurs boutons d'action. L'écran principal peut être doté d'un menu déroulant.

La création d'une application C# pour Windows comprend trois étapes majeures :

1. Dessiner l'écran (taille du form, choix et position des « contrôles » graphiques – boutons, zones de texte, boîtes de listes...).
2. Définir les propriétés de chaque contrôle graphique, dont leur nom.
3. Coder chacune des fonctions événementielles liées aux contrôles (sur simple-clic, faire ceci, au survol de souris, faire cela...).

Si l'application comporte plusieurs écrans ceci est à faire pour chaque écran.

Il est nécessaire de commencer par dessiner tous les écrans, en nommant correctement les contrôles, puis d'ajouter le code en le testant étape par étape.

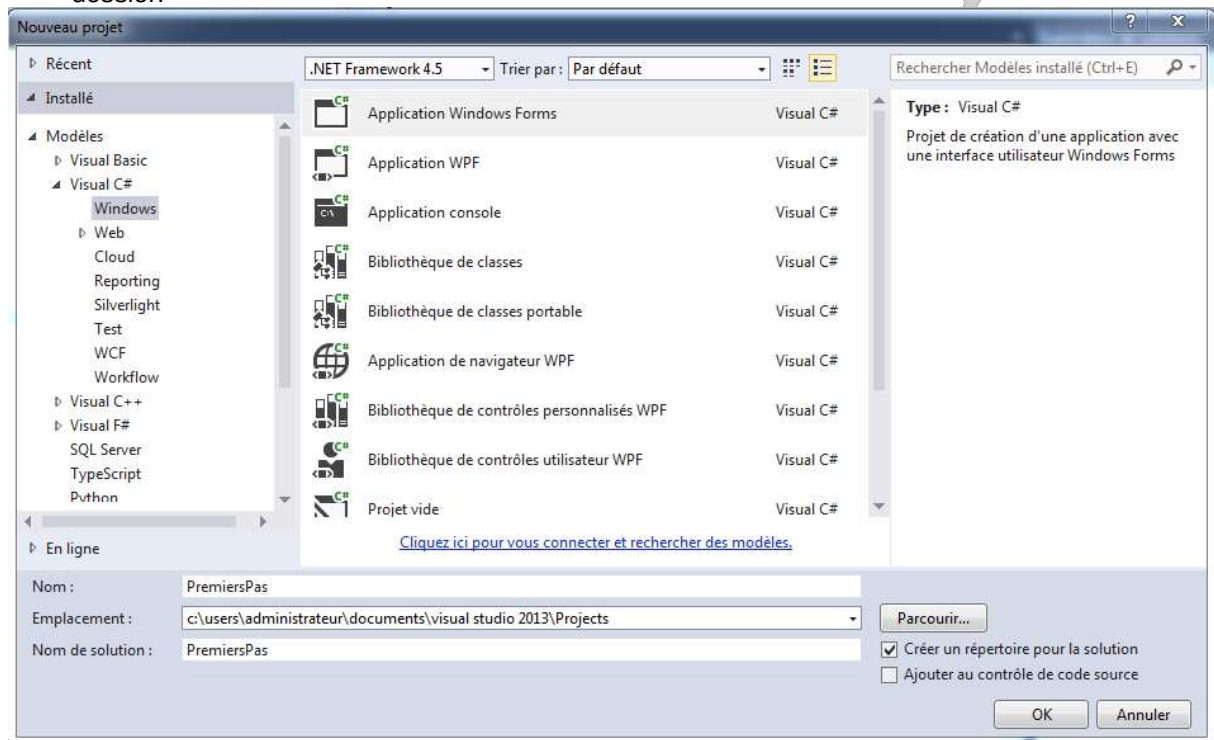
Une « bonne » interface utilisateur pour Windows doit répondre à de nombreux critères très précis, définis dans une charte graphique établie par Microsoft : menus déroulants, nombre de contrôles par fenêtre, utilisation des onglets, marges, encadrements, choix des types de contrôles, taille et position des boutons... Finalement, il n'est pas si facile pour le développeur de concevoir une bonne interface utilisateur Windows (d'autant plus que chaque version de Windows apporte son lot de nouvelles règles), et il faut retenir que l'utilisateur « jette » facilement une « mauvaise » application.

Etape 1 : Dessiner les écrans.

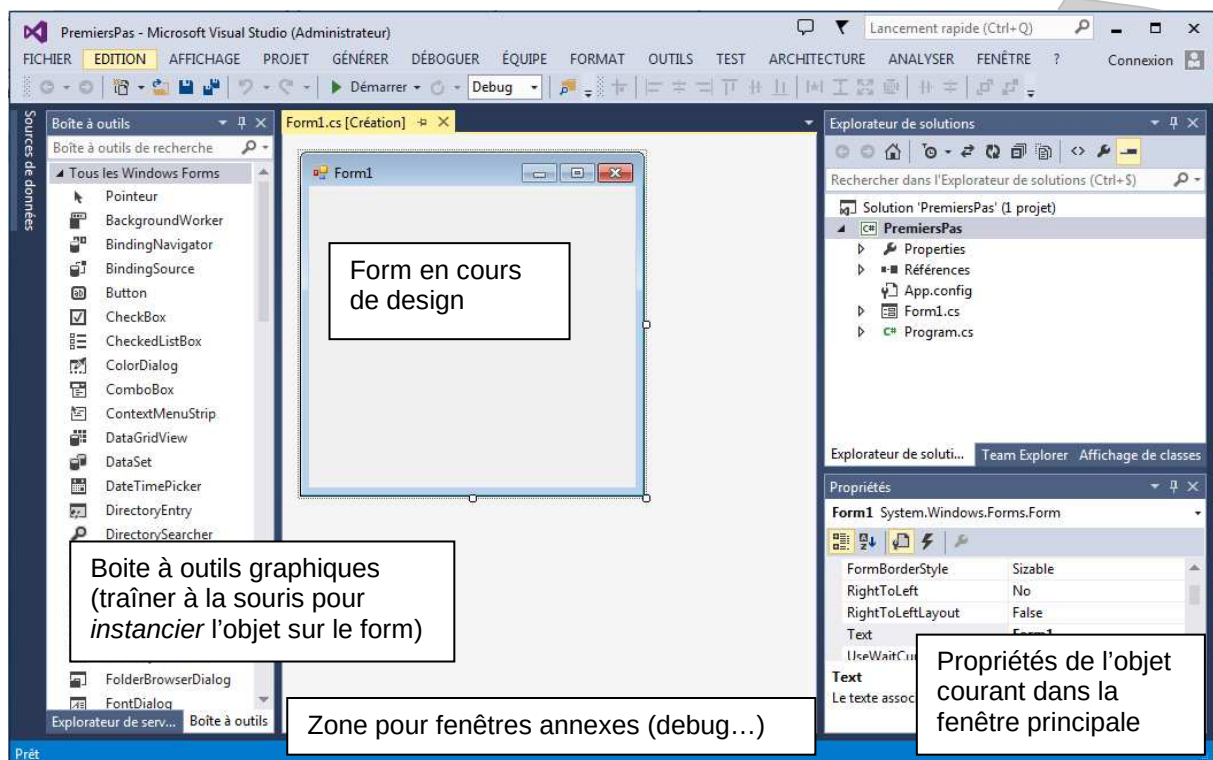
Présentation

Au lancement de Visual Studio 2013 choisir de créer un projet :

- Visual C#
- Application Windows Forms.
- Choisir pour nom de projet « PremiersPas »
- Accepter l'emplacement proposé ; Visual Studio créera tous les fichiers nécessaires dans ce dossier.



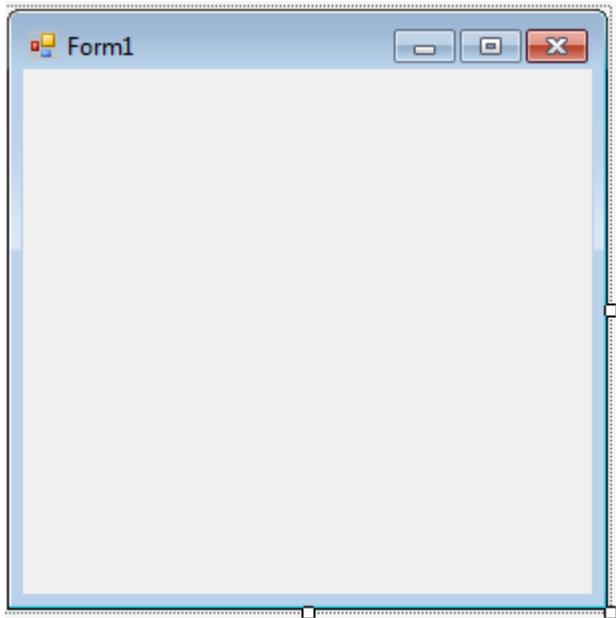
Voici l'espace de travail type d'un projet C# avec Visual Studio 2013 en cours de développement :



Attention : l'environnement de travail offert par Visual Studio est extrêmement riche et totalement paramétrable ; dans un premier temps, essayez de conserver les options d'affichage par défaut et de repérer ce dont vous avez besoin sans explorer l'ensemble des fonctionnalités. Assurez-vous que les fenêtres « Explorateur de solutions », « Propriétés » et boîte à outils » sont bien affichées (menu « Affichage ») comme dans l'exemple ci-dessus.

Feuille

La partie centrale permet de dessiner l'écran. Vous pouvez y insérer des contrôles graphiques (boutons, cases à cocher...), des éléments graphiques et des images pour donner à une feuille l'apparence que vous souhaitez. Au départ, elle est vide, comme ceci :



NB : dans ce document, les vocables :

- Ecrans
- Feuille
- Formulaire
- Form

seront synonymes.

Menu supérieur de Visual Studio.

Comme dans toute application Windows, le menu supérieur permet de gérer le projet, nous découvrirons les fonctions disponibles à mesure de nos besoins. Notez déjà :

- Le menu « Fichiers » qui permet de créer un « projet » puis de le sauvegarder, de fermer et d'ouvrir un projet ou une « solution » (vous pourrez au choix utiliser l'un ou l'autre de ces concepts puisque vos « solutions » ne comporteront qu'un seul « projet »).
- Le menu « Edition » qui permet de corriger du texte et qui regroupe, norme Windows oblige, les fonctions Couper/Copier/Collier et leurs raccourcis standards Ctrl-X/Ctrl-C/Ctrl-V.
- Le menu « Affichage » qui permet de définir quelles fenêtres doivent être ouvertes dans l'espace de travail (attention, l'écran est vite trop petit pour tout afficher ! Ne modifiez rien pour l'instant)
- Le menu « ? » qui donne des accès directs aux multiples fichiers d'aide en ligne chez Microsoft.

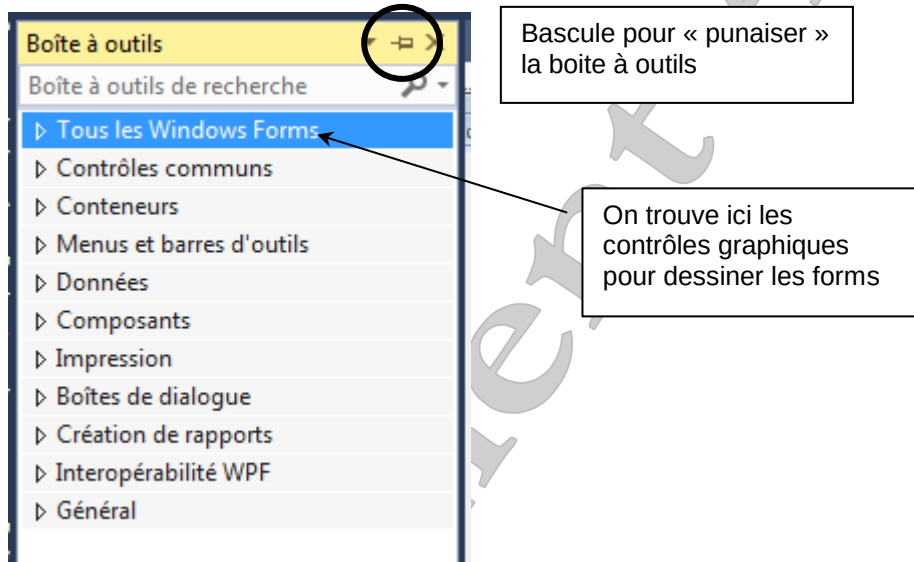
Boîte à outils.

Elle s'ouvre à partir du menu affichage. Par défaut, elle est ouverte et se place à gauche de

l'espace de travail. Vous pouvez la « punaiser » de manière à rendre son affichage permanent (mais cela prend de la place) ou la rendre discrète et auto-masquable.

La boîte à outils de Visual Studio contient les outils que vous pouvez utiliser pour dessiner des objets graphiques sur vos feuilles (onglet « Tous les Windows Forms ») et effectuer bien d'autres opérations (autres onglets). Chacun de ses outils représente un « contrôle », graphique ou non, prêt à l'emploi (on peut bien entendu enrichir la boîte à outils à l'aide de contrôles acquis ou développés...) ; les contrôles standards sont des éléments du .Net Framework et sont donc communs à tous les langages .Net.

Boîte à outils de Visual Studio 2013



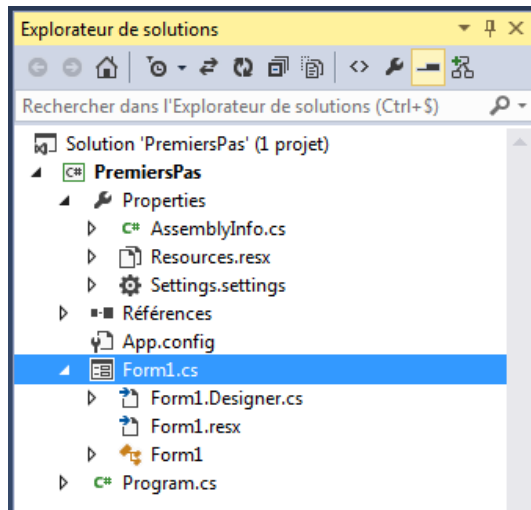
Le tableau suivant décrit brièvement les contrôles de base de C# figurant dans la boîte à outils.

Icône	Classe	Description
 Pointeur	Pointeur	Permet de déplacer et de redimensionner des feuilles et des contrôles (il s'agit plus d'un outil que d'un contrôle).
 PictureBox	PictureBox	Affiche des images en mode point (bitmaps), des icônes ou des métafichiers Microsoft Windows. Affiche aussi du texte ou agit comme un conteneur visuel pour d'autres contrôles.
 Label	Label	Affiche du texte que l'utilisateur ne peut pas modifier et avec lequel il ne peut pas dialoguer.
 TextBox	TextBox	Fournit une zone pour saisir ou afficher du texte.
 GroupBox	GroupBox	Constitue un conteneur visuel et fonctionnel pour des ensembles de contrôles (ensemble de boutons radio, de cases à cocher...).
 Button	Button	Exécute une commande ou une action lorsque l'utilisateur clique.

 CheckBox	CheckBox	ou « case à cocher » Représente une option Vrai/Faux ou Oui/Non. Le nombre de cases à cocher que vous pouvez activer simultanément est illimité.
 RadioButton	RadioButton	ou « bouton radio » Fait partie, avec d'autres boutons d'option, d'un groupe qui permet d'afficher plusieurs choix exclusifs les uns des autres. L'utilisateur ne peut sélectionner qu'un de ces boutons à la fois dans un groupe.
 ComboBox	ComboBox	Allie les fonctions d'une zone de texte et d'une zone de liste. Permet à l'utilisateur de taper un élément ou d'en sélectionner un dans une liste déroulante.
 ListBox	ListBox	Affiche une liste d'éléments parmi lesquels l'utilisateur peut faire un choix

Organisation du projet :

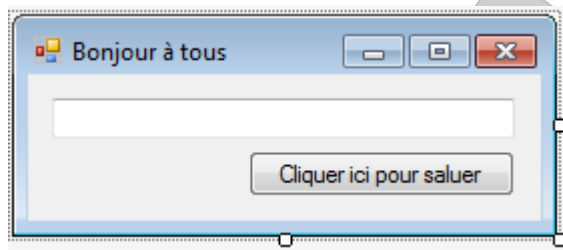
La fenêtre dans la partie droite de l'écran montre les fichiers du projet que vous pouvez être conduit à utiliser/modifier :



Vous y trouverez en particulier un fichier nommé « Form1.cs ». Il contiendra toute votre définition de cet écran, dessin et code. Prenons de suite de bonnes habitudes et renommons le : « frmPremiersPas.cs », par un clic droit.

Ecran à obtenir :

Notre écran « Premiers Pas » devra ressembler à ceci :



Cette mise en page vous paraîtra certainement très (trop) « tassée » ; elle est simplement conforme aux standards de design Windows ; pendant l'opération de design, Visual Studio vous aidera à respecter les alignements et positionnements préconisés.

Vous allez donc utiliser deux contrôles de la boîte à outils.

Contrôle	Nom
 TextBox	TextBox
 Button	CommandButton

Pour incorporer (« instancier ») un contrôle de la boîte à outils dans la feuille

1. Cliquez sur l'outil correspondant au contrôle que vous souhaitez incorporer (en l'occurrence, une zone de texte).
2. Déplacez le curseur sur votre feuille. Il prend la forme d'une croix.
3. Placez la croix à l'endroit où vous souhaitez voir apparaître le coin supérieur gauche du contrôle.
4. Faites glisser la croix jusqu'à ce que le contrôle atteigne la taille souhaitée. (Faire glisser signifie maintenir le bouton gauche de la souris enfoncé tout en utilisant celle-ci pour déplacer un objet.)
5. Relâchez le bouton de la souris. Le contrôle apparaît sur la feuille.

Vous pouvez également ajouter un contrôle à une feuille simplement en cliquant deux fois sur l'icône qui le représente dans la boîte à outils. Vous obtenez ainsi, en haut à gauche, de la feuille un contrôle ayant une taille par défaut qu'il reste à ajuster. Allez, dernière méthode, on peut encore trainer depuis la boîte à outils vers le form. A votre convenance...

Redimensionnement, déplacement et verrouillage des contrôles.

Remarquez les petites cases rectangulaires, nommées poignées de redimensionnement, situées dans les angles et médianes du contrôle. Vous les utiliserez lors de la prochaine étape pour redimensionner le contrôle. Un clic droit sur le contrôle permet d'accéder à un menu contextuel.

Pour redimensionner un contrôle

1. Sélectionnez le contrôle que vous souhaitez redimensionner en cliquant dessus à l'aide de la souris.
Des poignées de redimensionnement apparaissent sur le contrôle.
2. Positionnez le pointeur de la souris sur une poignée de redimensionnement et faites glisser la souris jusqu'à ce que le contrôle atteigne la taille souhaitée.
Les poignées situées dans les angles permettent de redimensionner les contrôles dans le sens de la hauteur et de la largeur, tandis que celles qui apparaissent sur les côtés autorisent le redimensionnement dans un seul sens uniquement.
3. Relâchez le bouton de la souris.
– Ou –
Utilisez la fenêtre Propriétés pour modifier la valeur des propriétés *Width* et *Height* du contrôle (dans *Size*).

Pour déplacer un contrôle

Utilisez la souris pour faire glisser le contrôle vers un nouvel emplacement sur la feuille.

– Ou –

Utilisez la fenêtre Propriétés pour modifier la valeur des propriétés *X* et *Y* (dans *Position*) du contrôle.

-Ou-

Utiliser la touche CTRL conjointement avec les touches de direction pour déplacer le contrôle d'une unité de quadrillage à la fois. Si le quadrillage est désactivé, le contrôle se déplace d'un pixel à la fois.

NB : vous remarquerez que lors des déplacements ou redimensionnements, la souris est « attirée » à des positions particulières, parfois matérialisées par des petits traits de cotes rouges ou bleus : il s'agit d'une aide au bon positionnement et à l'alignement des contrôles du form, de manière à faire aisément du « bon » Windows.

Pour verrouiller tous les contrôles dans leurs positions

Quand le design du form est terminé, il est prudent de figer taille et position des contrôles de manière à se prémunir contre des erreurs de manipulation. Cette opération permet de verrouiller tous les contrôles de la feuille dans leur position en cours afin d'éviter tout risque de déplacement accidentel. Seuls les contrôles figurant sur la feuille sélectionnée sont verrouillés ; cette opération n'a aucune incidence sur les contrôles des autres feuilles.

Menu **Format, Verrouiller les contrôles** ; cette commande est une bascule, de sorte que vous pouvez également l'utiliser pour déverrouiller les positions des contrôles.

Nous vous conseillons de verrouiller vos contrôles après la première étape de réalisation, le design des form.

Revenons à l'exercice

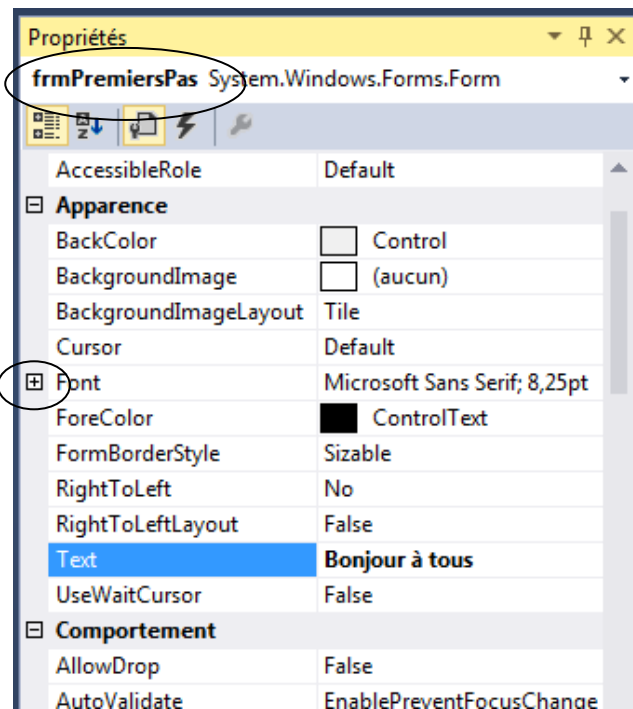
Créer selon le modèle un contrôle texte et un contrôle bouton de commande.

Etape 2 : Définir les propriétés de chaque contrôle.

Présentation

L'étape suivante consiste à définir les propriétés des objets que vous avez créés (« *instanciés* », plus exactement). La fenêtre *Propriétés* (illustrée dans la figure suivante) permet de définir simplement les propriétés de tous les objets d'une feuille et celles de la feuille elle-même. Pour ouvrir la fenêtre Propriétés, choisissez dans le menu **Affichage** la commande **Fenêtre**

Propriétés, cliquez sur le bouton  de la barre d'outils de l'Explorateur de solutions ou utilisez le menu contextuel associé au contrôle, ou pressez F4 (cette fenêtre est normalement ouverte et punaisée par défaut).



Fenêtre Propriétés.

La fenêtre Propriétés est constituée des éléments suivants :

En haut : la zone « Objet ». Elle affiche en gras le nom et le type de l'objet pour lequel vous pouvez définir des propriétés. Pour sélectionner un autre objet, affichez la liste des objets associés à la feuille en déroulant cette liste. A droite du nom de l'objet, Visual Studio indique son type. Dans le jargon de la programmation objet il s'agit de sa *classe*. Nous nous en inquièterons plus tard.

En dessous : la liste des propriétés. La colonne de gauche affiche les propriétés disponibles pour l'objet sélectionné. Certaines d'entre elles sont regroupées sous un nom générique : le signe  permet d'en voir le détail. Visual Studio offre 2 modes de tri des

propriétés : alphabétique  ou par catégorie . Vous pouvez consulter et modifier les valeurs dans la colonne de droite ; dans certains cas Visual Studio vous assiste en vous proposant une liste de valeurs possibles ou en vous offrant l'accès à un assistant ou une

autre boîte de dialogue().

Pour définir des propriétés à partir de la fenêtre Propriétés

1. Ouvrir la fenêtre des propriétés.
2. Dans la liste déroulante choisir le contrôle dont vous voulez modifier des propriétés
3. Dans la liste des propriétés, sélectionnez un nom de propriété.
4. Dans la colonne de droite, tapez ou sélectionnez la nouvelle valeur de la propriété.

Exercice

Dans le cas de l'exercice « Bonjour à tous ! », vous devrez changer la valeur de six propriétés. Laissez la valeur par défaut pour toutes les autres propriétés.

Prenez dès maintenant l'habitude de renommer chaque contrôle en le préfixant selon son type. Par convention, dans notre équipe de travail, les noms de chaque objet commencent par trois lettres minuscules évoquant leur type.

Objet	Propriété	Valeur
Feuille	Name	frmPremiersPas
	Text	Premiers pas : Bonjour à tous
Zone de texte	Name	txtBonjour
	Text	(vide)
Bouton de commande	Name	btnBonjour
	Text	Cliquer ici pour saluer

Notez que toutes les valeurs que vous avez modifiées apparaissent en caractères gras

Etape 3 : Coder les procédures événementielles

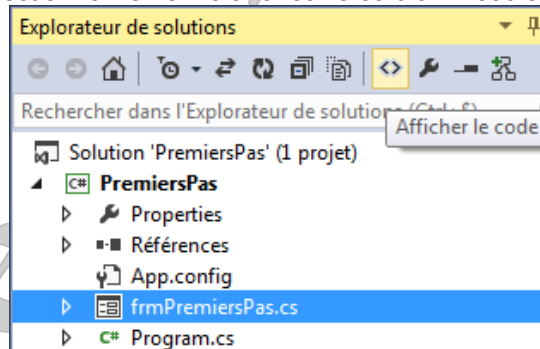
Présentation.

Le code C# de votre application s'écrit dans la fenêtre *Code*. Un code est constitué d'instructions, de constantes et de déclarations. La fenêtre Code vous permet de visualiser et de modifier rapidement n'importe quelle partie du code de votre application.

Pour ouvrir la fenêtre Code

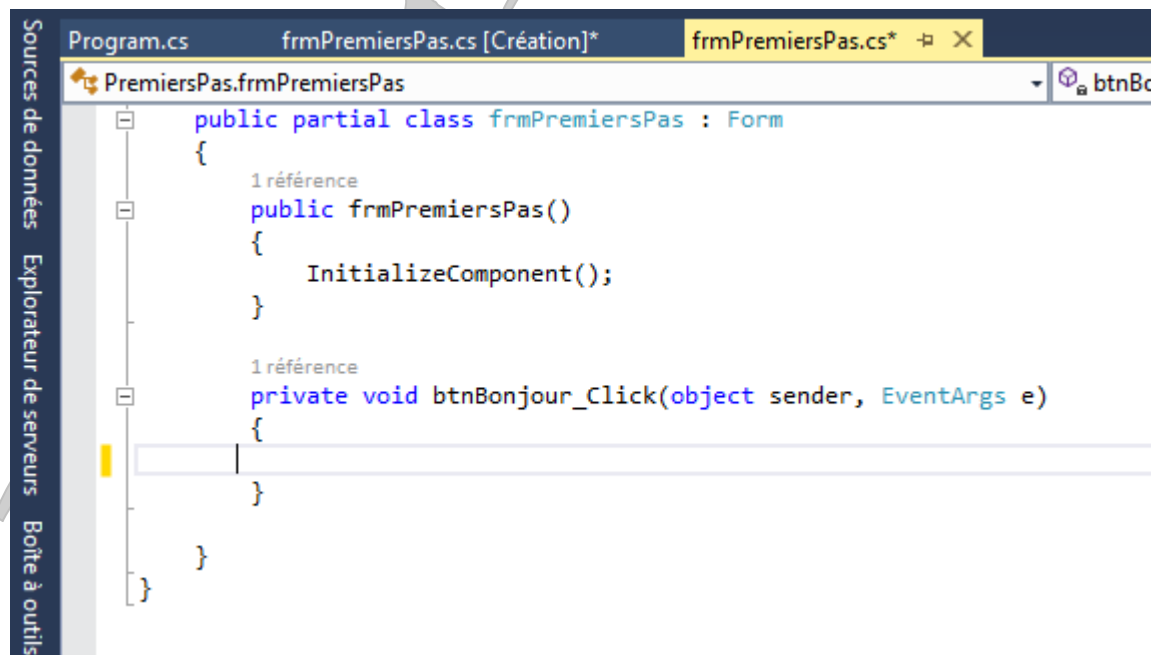
Dans la fenêtre principale de design, sélectionnez l'objet graphique et double-cliquez dessus
ou
frappez F7 ou clic-droit/*Afficher le code*
ou

Dans la fenêtre Explorateur de solutions, sélectionnez le nom d'une feuille ou d'un module,



puis choisissez le bouton « *Afficher le code* ».

Double-cliquez maintenant sur le bouton du form : la fenêtre de code suivante apparaît :



Remarquer la présence des deux onglets :

- frmPremiersPas.cs [Création] → la partie visuelle, graphique du form
- frmPremiersPas.cs → la partie code du form

Bien que vous n'ayez encore rien saisi, la fenêtre code n'est pas vide, elle contient :

- une déclaration des ressources utilisées par ce code (entre autres, la bibliothèque System.Windows.Forms du .Net Framework qui fournit le « moule » à fabriquer des feuilles.
- Une définition de classe, en effet pour C# votre écran est une classe, orientation objet oblige.
- Un constructeur de la classe (« public frmPremiersPas() {...} ») ; nous le personnalisons ultérieurement.

Deux listes déroulantes apparaissent dans la partie haute de la fenêtre ; elles vous aideront à vous repérer dans les différentes portions de code C# disséminés dans le projet. Le curseur clignote après **private void btnBonjour_Click(...)** pour vous inviter à saisir le code correspondant aux actions que vous souhaitez déclencher quand l'utilisateur clique sur ce bouton.

La fenêtre *Explorateur de Solution* liste un autre composant de votre form :

frmPremiersPas.Designer.cs ; double-cliquez pour l'ouvrir : il contient le code C# nécessaire pour dessiner votre form et ses 2 contrôles ; ce code a été généré par Visual Studio lors du design graphique de votre écran réalisé dans le mode d'affichage précédent *Création*. Observez en particulier le contenu du pavé « code généré par le concepteur Windows Form » .

- **Il est fortement déconseillé de modifier ce code généré, mais on peut regarder.**

Revenons à la feuille de code frmPremiersPas.cs ; travaillons proprement et ajoutons en tête un élément de documentation pour résumer ce que l'application est censée faire :

Placez le curseur juste avant le « p » de **public partial class frmPremiersPas : Form** et frappez 3 caractères slash (/) ; Visual Studio génère pour vous un embryon de documentation (`/// <summary>`) qu'il vous reste à compléter :

```
/// <summary>
/// Application "premiers pas"
/// Auteur xxx, le jj/mm/aaaa
///
/// Cette application écrit à la demande « bonjour à tous » dans une zone de texte.
/// C'est donc une première illustration de la programmation événementielle.
/// Cette application simplissime permet déjà de montrer les principaux
/// concepts de C#. */
/// </summary>
public partial class frmPremiersPas : Form
```

Code événementiel

Notre application, développée pour Windows, doit pouvoir répondre aux différentes sollicitations de l'utilisateur : clic sur un bouton, choix dans une liste déroulante, frappe de caractères dans une zone de texte, passage d'une zone à l'autre, survol d'un contrôle par la souris... Le système d'exploitation Windows détecte chaque interaction de l'utilisateur dans la fenêtre active et en informe l'application concernée ; au programmeur de coder les actions correspondantes. On parle de programmation événementielle en ce sens que le développeur regroupe les instructions à exécuter dans de multiples sous-programmes (procédures événementielles) correspondant chacun à un événement à intercepter ; le programme principal n'est même plus à écrire (et Visual Studio a généré le nécessaire au démarrage) ; il est simplement représenté par l'affichage du form lui-même qui inclut une attente perpétuelle d'événements.

Une *procédure d'événement*, comme celles que vous allez personnaliser ici, contient donc un code qui s'exécute lorsqu'un événement se produit au moment de l'exécution (si l'utilisateur clique sur le bouton, dans notre exemple).

La fenêtre code contient déjà :

```
private void btnBonjour_Click(object sender, EventArgs e)
{
}

```

Les paramètres fournis en entrée peuvent vous aider à connaître, dans le code, le contexte de l'événement :

- "sender" est l'objet à l'origine de l'événement.
- "e" comporte des détails sur l'événement.

Pour l'événement `btnBonjour.Click`, la procédure est nommée `btnBonjour_Click()`. Par défaut Visual Studio lui donne ce nom **et** la relie à l'événement (par une instruction générée et 'cachée' dans la partie de code `.designer.cs`).

Ajouter donc un commentaire pour spécifier ce que la procédure devra faire et précisons l'unique instruction nécessaire :

```
private void btnBonjour_Click(object sender, EventArgs e)
{
    /* Le traitement consiste à recopier la chaîne "bonjour ..."
     * dans l'objet txtBonjour du formulaire courant (this)
     * la propriété affectée est évidemment "text"
     */
    this.txtBonjour.Text = "Bonjour à tous !";
}

```

this désigne la référence au formulaire courant ; lorsque vous tapez le point qui suit le mot `this`, Visual Studio vous propose une liste déroulante contenant la liste des contrôles, propriétés et méthodes de ce form ; choisissez donc « `txtBonjour` », frappez le point et Visual Studio vous propose la liste des propriétés et méthodes de ce contrôle ; choisissez donc `Text`.

Ce processus d'assistance à l'écriture du code, nommé IntelliSense par Microsoft, est très fiable et bien pratique pour le développeur et évite de nombreuses fautes de frappe.

Attention : C# tient compte des majuscules et minuscules ! `txtBonjour` n'est pas la même chose que `TXTBonjour`.

Vous avez maintenant saisi 2 types de commentaires dans votre code :

- le commentaire pur (`/* ... */`) à destination du programmeur qui relit le code source
- la documentation (`///`) qui permettrait de générer des pages HTML d'aide sur l'application sans toutefois fournir le code source

Nous aurons l'occasion d'approfondir ces notions ultérieurement.

Globalement la page de code ressemblera à ceci :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;

namespace PremiersPas
{
    /// <summary>
    /// Application "premiers pas"
    /// Auteur xxx, le jj/mm/aaaa
    ///
    /// Cette application écrit à la demande « bonjour à tous » dans une zone de texte.
    /// C'est donc une première illustration de la programmation événementielle.
    /// Cette application simplissime permet déjà de montrer les principaux
    /// concepts de C#. */
    /// </summary>
    public partial class frmPremiersPas : Form
    {
        public frmPremiersPas()
        {
            InitializeComponent();
        }

        private void btnBonjour_Click(object sender, EventArgs e)
        {
            /* Le traitement consiste à recopier la chaîne "bonjour ..."
            * dans l'objet txtBonjour du formulaire courant (this)
            * la propriété affectée est évidemment "text"
            */
            this.txtBonjour.Text = "Bonjour à tous !";
        }
    }
}
```

Ressources utilisées par ce programme

/// commentaire multilignes de documentation

Classe partielle : elle est définie par 2 pages de code C# (fichier .cs et fichier .designer.cs)

« constructeur » de la classe

/* commentaire multilignes */

Procédure événementielle : le code « utile » et commenté, celui que vous avez ajouté pour effectuer les traitements voulus

Notez qu'au final nous avons écrit plus de commentaires que d'instructions car C# nous permet de manipuler très simplement les propriétés des objets définis en mode Création dans nos programmes. Toutefois, comme pour un iceberg, vous ne voyez actuellement que la partie « émergée » du programme ; il vous a suffi d'ouvrir la feuille de code complémentaire et sa *région générée par le concepteur Windows* pour prendre toute la mesure du code nécessaire, et tout l'intérêt d'utiliser ce *mode Création* de Visual Studio.

Lancer l'application et contrôler les résultats

Vous devez d'abord *générer* le projet. Cela provoquera la génération du code intermédiaire (ou « MicroSoft Intermediate Language » identique pour tous les langages sources .Net et exécutable par le « Common Language Runtime » fourni avec le .Net Framework ; un peu compliqué tout cela mais on peut ignorer l'architecture interne tant que l'on développe des projets simples avec un seul langage).

Pour ce faire utilisez le menu « GENERER ». Vous pourrez regarder avec l'explorateur de fichiers de Windows, les différents fichiers générés dans le dossier de votre projet.

N'oubliez pas de sauvegarder votre projet : utilisez le menu FICHIER/Enregistrer tout ou l'icône disquette.

Pour exécuter l'application, vous pouvez au choix :

- Cliquer sur la flèche de débogage dans la barre d'outils



- Frapper F5
- Aller dans le menu DEBOGUER

Après un court instant de compilation vous verrez votre écran apparaître ; il s'agit bien d'un programme en cours d'exécution, indépendant de Visual Studio. Le module Program.cs généré par Visual Studio a lancé l'exécution de votre form.

Cliquez sur le bouton de commande que vous avez créé sur le form, et vous verrez la phrase « Bonjour à tous ! » s'afficher dans la zone de texte.

Vous venez de créer votre première application fenêtrée et de gérer un événement.

Bravo, continuez, toute la suite sera sur le même principe.

Synthèse ; les points-clés de l'exercice

- Un projet C# .Net regroupe un ensemble de fichiers dont les form qui contrôlent le dialogue avec l'utilisateur
- Visual Studio offre une palette d'outils permettant d'instancier des objets graphiques prêts à l'emploi (zones de texte, listes déroulantes, cases à cocher...)
- Le mode Création permet au développeur de mettre en page aisément les écrans de dialogue avec l'utilisateur ; son rôle est avant tout de générer le code C# correspondant à la mise en page souhaitée
- Un form C# contient deux parties distinctes en interaction étroite :
 - l'interface graphique (mode Créatio)
 - le code (bouton Afficher le code)
- Un développement C# consiste à :
 1. dessiner les form en plaçant les contrôles graphiques nécessaires au dialogue,
 2. ajuster les valeurs initiales des propriétés et nommer correctement chaque objet (contrôle graphique, form...),
 3. écrire le code C# associé à chaque événement autorisé sur chacun des contrôles graphiques
- La programmation en .Net est basée sur l'utilisation (récupération, affectation...) des valeurs de propriétés des objets constituant l'application
- Un projet C# peut comporter de multiples form ; l'un d'eux, ou une procédure de code, doit être désigné comme l'objet à activer au démarrage
- Eléments de syntaxe C# :
 - Une instruction par ligne (la fin d'instruction est le caractère point-virgule)
 - Un commentaire commence par // s'il tient sur une seule ligne, ou il est encadré par /* ... */
 - Un commentaire de documentation commence par /// juste avant le membre de la classe à documenter
 - Un bloc (de procédure, de fonction, de classe, de condition...) est défini par une suite de mots (private, class, if...) et son contenu est encadré par les caractères { et }
 - Visual Studio génère le code C# nécessaire au design des écrans ; Visual Studio apporte aussi une aide à l'écriture manuelle du code (indentation automatique, contrôle syntaxique en cours de saisie, liste de choix contextuelle pour les mots-clés appelée « IntelliSense »...)